

Homework — 1.2.3 Software Development — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. **[2]** Analysis → Design → Development/Implementation → Testing → Evaluation. (2 marks if all five in correct order; 1 mark if mostly correct with one slip.)
2. **[2]** Any two (1 each): investigate the current problem/system; gather and document requirements (e.g. interviews, observation); assess feasibility (technical, economic, time); define the scope/objectives of the system.
3. **[3]** Two XP practices (1 each, max 2): pair programming (two developers at one workstation); test-driven development / constant testing; the customer is part of the development team; frequent small releases; collective code ownership/refactoring. Suited to (1): projects where high code quality/reliability is essential, or where requirements change frequently and close customer involvement is possible. Max 3.
4. **[4]** Boundary: 0 or 100 (also accept -1/101 as just-outside boundary) (1); Normal/valid: any value in range e.g. 57 (1); Erroneous: a value that should be rejected e.g. 150 or "abc" (1). Boundary testing is important because errors most often occur at the edges of the accepted range (e.g. using > instead of >=), so testing 0 and 100 checks the limits are handled correctly (1).
5. **[3]** Any three (1 each): effective (meets the requirements / does the job); usable (easy to use / good user interface); robust (handles invalid input / does not crash); maintainable (well structured/documented so it can be changed); also accept efficient/reliable. Max 3.

Part B (6 marks)

6. **[2]** Any two (1 each): compiler translates all at once before running, interpreter translates and executes line by line at run time; compiler produces a standalone executable, interpreter produces none; compiled code runs faster; interpreted code needs the source + interpreter present each run; interpreter stops at the first error, compiler reports errors at the end. Max 2.
7. **[2]** Washing machine → embedded OS, because it performs one dedicated task with limited resources, is stored in ROM and must be reliable/efficient (1). Server cluster → distributed OS, because it runs across multiple networked computers as one system, sharing resources/workload (1).
8. **[2]** RAM is volatile — loses its contents when power is removed — and holds the programs and data currently in use (1). ROM is non-volatile — keeps its contents without power — and stores firmware such as the BIOS/boot program (1).

Total: 14 + 6 = 20